YENEPOYA UNIVERSITY

HIGH LEVEL DESIGN (HLD) DOCUMENT
ON
AI-Enhanced CI/CD for Smart City Software

Student Name: Suzana Sehanaz

Registration No: 23BCAICD104

Campus ID: 27019

Project Code: PRJN26-045

Industry Mentor: Sumit Kumar Shukla

Guided By: Ms. Deeksha Shetty

TABLE OF CONTENTS

1. Introduction
   - 1.1 Scope of the Document
   - 1.2 Intended Audience
   - 1.3 System Overview
2. System Design
   - 2.1 Application Architecture
   - 2.2 Process Flow
   - 2.3 Information Flow
   - 2.4 Components Design
   - 2.5 Key Design Considerations
   - 2.6 Technology Stack
3. Data Design
   - 3.1 Data Model
   - 3.2 Data Access Mechanism
   - 3.3 Data Retention Policies
4. Interfaces
5. State and Session Management
6. Caching Strategy
7. Non-Functional Requirements
   - 7.1 Performance Aspects
   - 7.2 Scalability
8. References

1. Introduction

This document presents the High Level Design for the Smart City CI/CD Pipeline project. The project is a Python-based system that calculates Smart City performance metrics and uses GitHub Actions to automatically run all tests every time code is pushed. The goal was to understand how professional software teams enforce code quality through continuous integration.

2. Scope of the Document

- Covers the overall architecture, major components, and their interactions
- Describes the CI/CD automation flow and its triggers
- Explains data flow, interface design, and non-functional requirements
- Does not cover line-by-line code details — those are in the LLD

3. Intended Audience

- Industry Mentor — Sumith K Shukla
- Faculty guide and academic evaluators at Yenepoya Institute
- Fellow students looking to understand CI/CD architecture

4. System Overview

The Smart City CI/CD Pipeline consists of three Python functions that calculate city performance metrics, a 15-test unit test suite, and a GitHub Actions workflow that runs all tests automatically on every code push across three Python versions.

Component	Technology	Role
smart_city.py	Python 3.x	Core calculation functions
test_smart_city.py	Python unittest	15 unit tests covering all functions
.github/workflows/ci.yml	GitHub Actions (YAML)	Runs tests on every push automatically
GitHub Repository	Git / GitHub	Version control and CI/CD host

5. System Design

5.1 Application Design

The system follows a three-layer separation: source code, tests, and automation. Each layer is independent and modifiable without affecting the others.

```
+------------------+------------------+------------------+
|  SOURCE CODE     |  TEST SUITE      |  AUTOMATION      |
|                  |                  |                  |
|  smart_city.py   | test_smart_      | .github/         |
|  -------------   | city.py          | workflows/ci.yml |
|  Energy Func     | -------------    | -------------    |
|  Traffic Func    | 5 Energy Tests   | on: push / PR    |
|  AQI Func        | 5 Traffic Tests  | Python 3.10/11/12|
|                  | 5 AQI Tests      | Ubuntu VM        |
|                  | Total: 15 Tests  |                  |
+------------------+------------------+------------------+
```

5.2 Process Flow

```
Developer
   |
   |  Write / edit code locally
   |  Run tests locally to verify
   |
   |  git commit && git push
   v
GitHub Repository
   |
   |  GitHub Actions auto-triggered
   v
+-------------------------------------+
|  GitHub Actions Runner (Ubuntu)     |
|  Step 1: Checkout repository        |
|  Step 2: Setup Python 3.10/11/12    |
|  Step 3: Run 15 unit tests          |
+----------------+-------------------+
                 |
         +-------+-------+
         |               |
      PASS (OK)       FAIL (error)
         |               |
    Green badge      Red badge
    Merge allowed    Fix required
```

5.3 Information Flow

```
Input Parameters                Function               Output
-----------------               --------               ------
consumption_kwh, population  -> energy_efficiency()  -> float
vehicles, road_capacity      -> traffic_congestion() -> float
pm25, pm10, no2              -> air_quality_index()  -> dict
All outputs  ->  Unit Tests (assertEqual / assertRaises)
Test results ->  GitHub Actions  ->  Green / Red badge
```

5.4 Components Design

Component	Responsibility	Key Contents	Depends On
smart_city.py	Implements all calculation functions	3 functions with input validation	Python stdlib only
test_smart_city.py	Verifies all functions across 15 scenarios	3 TestCase classes, 15 test methods	smart_city.py, unittest
ci.yml	Automates test execution on every push	Trigger config, matrix, steps	GitHub Actions, test file

5.5 Key Design Considerations

- No external dependencies — uses Python standard library only
- Matrix testing across Python 3.10, 3.11, and 3.12 in parallel
- Edge-case-first test design — zero values, negatives, over-capacity
- Full CI run completes in under 20 seconds
- Stateless functions — no shared state between calls

5.6 API Catalogue

Function	Returns	Purpose
calculate_energy_efficiency(kwh, pop)	float	Energy consumption per capita (kWh/person)
calculate_traffic_congestion_index(v, cap)	float	Congestion ratio — above 1.0 means gridlock
calculate_air_quality_index(pm25, pm10, no2)	dict	AQI score + health category string

6. Data Design

Data Diagram

```
                                      +---------------+
+-------------+                       |  crop_status  |
| User Message|   input               |  dictionary   |
| current     |----------+            +---------------+
| input text  |          |  crop      +---------------+
+-------------+          |  topic     |  soil_info    |
                         v            |  dictionary   |
                +----------------+    +---------------+
+-------------+ | command_       |-> +---------------+
| History List| | handler.py     |    |  fertilizer   |
| previous    |-| get_response() |    |  dictionary   |
| commands    | +----------------+    +---------------+
+-------------+          |            +---------------+
                         | weather/   | weather_tips |
                         | irr        | list          |
                         +---------> +---------------+
                                      +---------------+
                                      |  utilities    |
                                      |  time/random  |
                                      +---------------+
```

6.1 Data Model

Field	Type	Unit	Range	Description
consumption_kwh	float	kWh	> 0	Total energy in city zone
population	int	persons	> 0	Residents in the zone
vehicles	int	count	>= 0	Vehicles on road currently
road_capacity	int	count	> 0	Max vehicle capacity of road
pm25 / pm10 / no2	float	µg/m³	>= 0	Pollutant concentration levels

6.2 Data Access Mechanism

- All data passed directly as function arguments — no database or ORM
- Test data defined as hardcoded literals within each test method
- In a real deployment, functions would receive data from IoT sensor feeds

6.3 Data Retention Policies

- No persistent storage — system operates entirely in memory
- XML test result artefacts retained on GitHub Actions for 90 days (default)

6.4 Data Migration

Not applicable. No database is used at this stage.

7. Interfaces

Interface	Type	Direction	Description
GitHub Repository	Git over HTTPS	Outbound	Developer pushes commits to remote
GitHub Actions Runner	GitHub-hosted Ubuntu VM	Inbound trigger	Auto-triggered on push to run tests
Developer Terminal	Command line (local)	Two-way	Run tests locally, interact with Git
Artefact Store	GitHub REST API	Outbound	XML test results uploaded after each run

8. State and Session Management

- System is fully stateless — each function call is independent
- No global variables or shared state between calls
- Each test method runs in isolation — setUp/tearDown handled by unittest
- GitHub Actions runner is ephemeral — fresh VM per workflow run

9. Caching

- No caching implemented — functions are computationally lightweight (microseconds)
- Python package caching via actions/cache could be added if third-party packages are introduced
- Current setup uses only standard library, so caching is unnecessary

10. Non-Functional Requirements

10.1 Security Aspects

- No secrets or credentials hardcoded in any source file
- Pinned action versions (checkout@v4, setup-python@v5) to prevent supply chain attacks
- No external user input — eliminates injection attack vectors
- GitHub branch protection rules can block merges that fail CI

10.2 Performance Aspects

- Each function executes in under 1 millisecond
- Full test suite (15 tests) completes in ~2–3ms locally
- Full GitHub Actions CI run completes in under 20 seconds per Python version
- All 3 Python versions run in parallel — no sequential bottleneck

11. References

#	Source	Link / Description
1	Python unittest	https://docs.python.org/3/library/unittest.html
2	GitHub Actions	https://docs.github.com/en/actions
3	actions/checkout@v4	https://github.com/actions/checkout
4	actions/setup-python@v5	https://github.com/actions/setup-python
5	xmlrunner	https://pypi.org/project/xmlrunner/